



MANUAL TÉCNICO

TURMA INI3A - EQUIPE 4

- 01 - Ana Carolina Tiritan
- 11 - Guilherme Albaricci Alves
- 18 - Lucas Dalben Borges
- 21 - Maria Eduarda Batista Ribeiro
- 24 - Matheus Batista de Souza
- 25 - Nicole Corrêa Colasso

SUMÁRIO

1. CONTEXTUALIZAÇÃO	3
1.1 Objetivo do Projeto	3
1.2 Motivação e Problema Identificado	3
1.3 Solução Proposta	3
2. DIAGRAMAS	4
2.1 Diagrama de Classes	4
2.2 Diagrama de Banco de Dados	6
3. TECNOLOGIAS UTILIZADAS	8
3.1 Padrão de Arquitetura	8
3.2 Linguagens e Tecnologias Front-End	8
3.3 Linguagens, Frameworks e Infraestrutura Back-End	8
3.4 Sistema de Banco de Dados (SGBD)	8
3.5 Integrações e APIs Externas	9
3.6 Ambientes e Ferramentas de Implementação e Controle	9
4. SETUP DO SISTEMA EM PRODUÇÃO	9
4.1 Pré-Requisitos	9
4.2 Acesso ao Servidor	11
4.3 Instalação das dependências PHP	12
4.4 Configurações do .env	12
4.5 Cópia dos arquivos	14
4.6 Prefixo das rotas	14
4.7 Criação das tabelas no Banco de Dados	15
4.8 Permissões de escrita	16
4.9 Configurações do Apache	17
4.10 Publicação da API de Whatsapp	18
5. SETUP DE DESENVOLVIMENTO	19
5.1 Pré-requisitos	19
5.2 Obtenção do código	20
5.3 Instalação das dependências PHP	21
5.4 Criação do banco de dados local	21
5.5 Configurações do arquivo .env	22
5.6 Criação das tabelas e dados iniciais	23
5.7 Execução da API de Whatsapp	24
5.8 Execução do sistema	25
5.9 Estrutura do projeto	25

1. CONTEXTUALIZAÇÃO

1.1 Objetivo do Projeto

O Brecharme é uma plataforma web desenvolvida para modernizar, digitalizar e otimizar a gestão do brechó solidário mantido pela Cáritas Bauru. O objetivo principal do sistema é intermediar o fluxo de doações de peças de vestuário e utensílios, simplificar a triagem e organizar o catálogo de produtos, permitindo que a comunidade visualize, consulte e reserve itens de forma simples, rápida e acessível, além de facilitar a divulgação e comunicação dos administradores do bazar com o público.

1.2 Motivação e Problema Identificado

Tradicionalmente, a gestão de brechós solidários enfrenta desafios operacionais significativos, tais como:

- Controle manual e descentralizado: O registo de doações e a gestão de estoque em papel ou folhas de cálculo dificultavam a triagem e o acompanhamento dos produtos disponíveis.
- Baixa visibilidade do estoque: A comunidade local só conseguia conhecer as peças disponíveis deslocando-se presencialmente ao espaço físico, o que limitava o alcance das vendas solidárias e a arrecadação de recursos para as ações sociais da instituição.
- Comunicação lenta: A falta de um canal automatizado para confirmação de reservas ou agendamento de entregas gerava ruídos de comunicação entre doadores, compradores e a equipe voluntária.

1.3 Solução Proposta

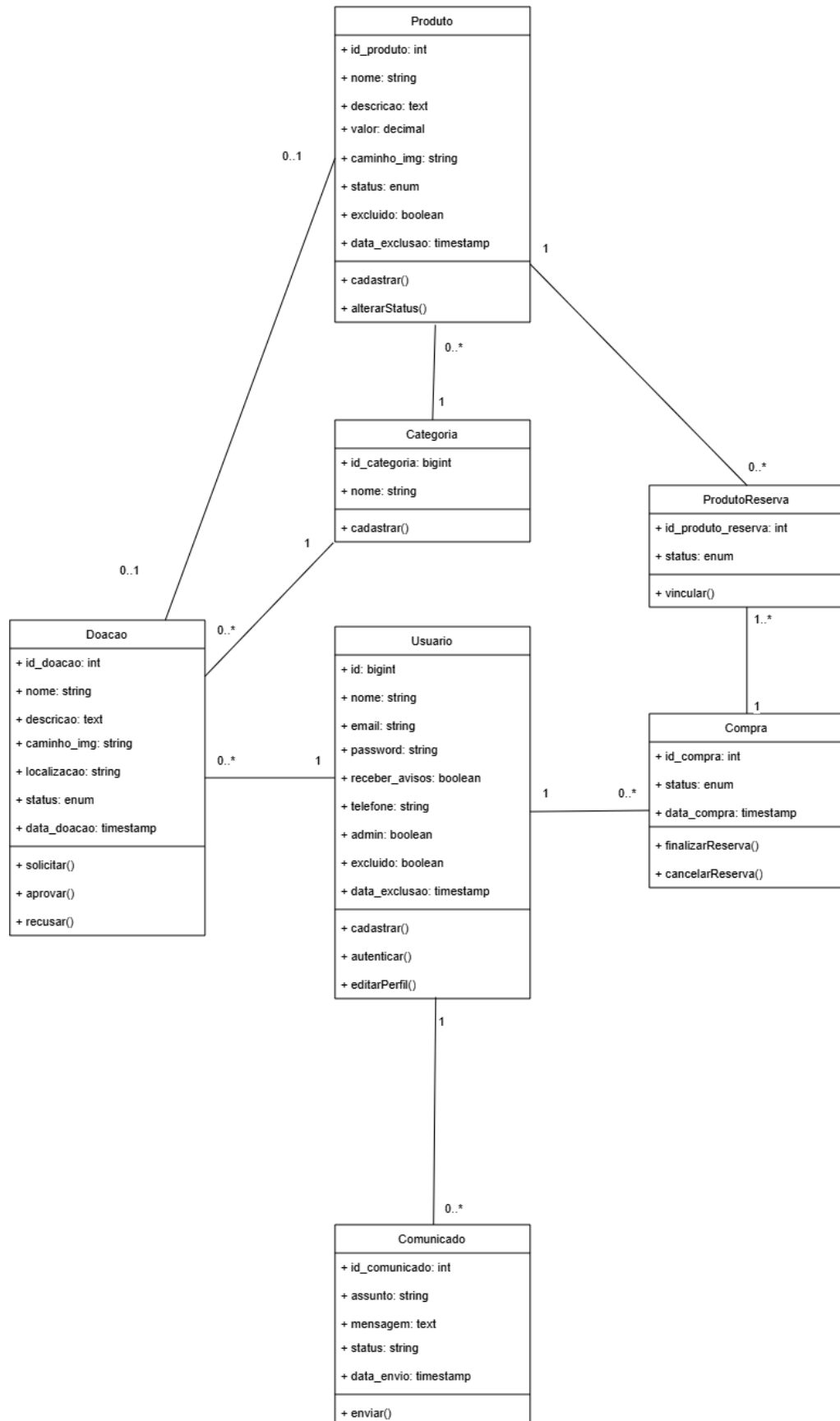
A plataforma Brecharme resolve estas limitações ao integrar num único ecossistema digital:

- Gestão prática de doações: Um módulo dedicado para que doadores submetam itens para aprovação e triagem da equipa administrativa.
- Catálogo e reserva online: Uma vitrina digital onde os utilizadores podem explorar o catálogo de produtos e realizar a reserva direta das suas peças de interesse.
- Notificações automatizadas: Integração com serviços de email e mensagens instantâneas para confirmar reservas e doações e manter o usuário informado sobre as novidades do Brecharme.

Com esta solução, o Brecharme atua como uma ponte digital entre a solidariedade da comunidade de Bauru e o trabalho social promovido pela Cáritas Bauru.

2. DIAGRAMAS

2.1 Diagrama de Classes



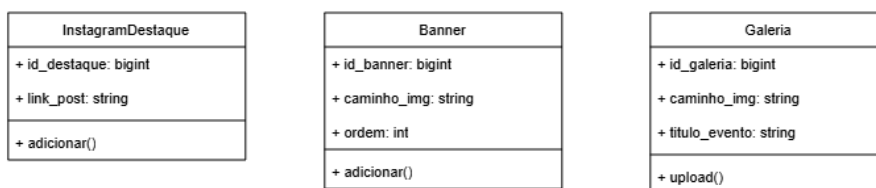


Imagem: Diagrama de Classes

O Diagrama de Classes descreve a estrutura estática da aplicação Brecharme, ilustrando suas entidades, atributos, métodos e os relacionamentos do domínio do sistema. O modelo divide-se nos seguintes módulos principais:

- **Gestão de Usuários e Comunicação:**
 - **Usuario:** Entidade central da aplicação, responsável por armazenar dados de cadastro, autenticação e permissões de acesso (admin). Relaciona-se diretamente com doações, compras e comunicados.
 - **Comunicado:** Representa os avisos e mensagens enviados aos usuários pelo administrador (1:0...*).
- **Gestão de Doações e Produtos:**
 - **Doacao:** Registra as solicitações de doações realizadas pelos usuários. Uma doação aprovada pode originar um item do estoque (Produto).
 - **Produto:** Representa os itens cadastrados no brechó para venda. Cada produto possui atributos como preço, descrição, status e vinculação a uma Categoria (0...*:1).
- **Módulo de Reservas e Compras:**
 - **Compra:** Gerencia o processo de checkout/reserva de um usuário (1:0...*).
 - **ProdutoReserva:** Classe associativa que conecta os produtos selecionados a uma compra específica (1...*:1).
- **Módulo Institucional e Conteúdo da Plataforma:**
 - **InstagramDestaque, Banner e Galeria:** Classes independentes responsáveis pelo gerenciamento de mídias visuais, destaques de redes sociais e banners da interface da plataforma.



2.2 Diagrama de Banco de Dados

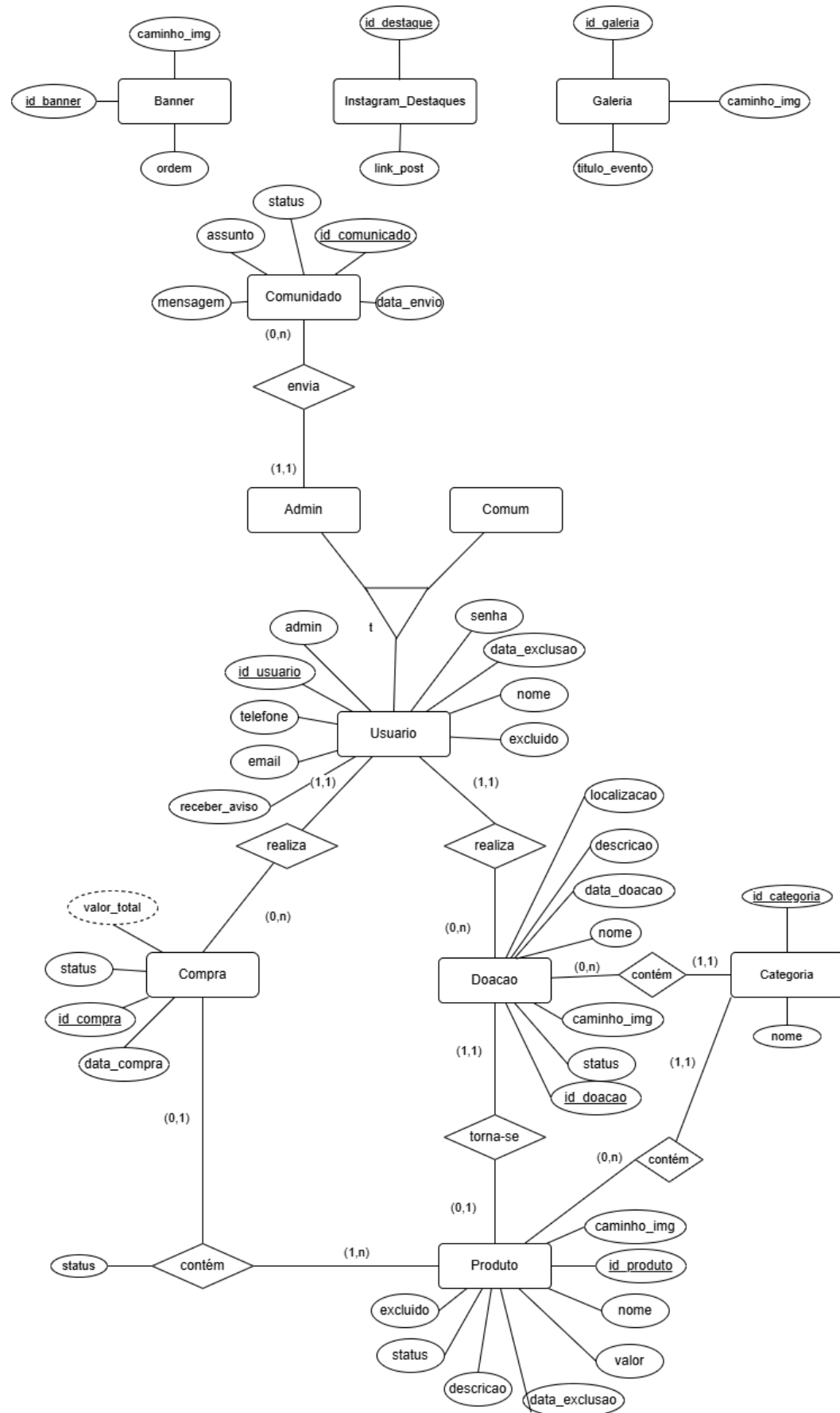


Imagem: Diagrama de Banco de Dados



O Diagrama de Banco de Dados no modelo Entidade-Relacionamento representa a modelagem conceitual do banco de dados relacional da aplicação Brecharme. O diagrama mapeia as entidades do sistema, seus atributos chave e derivados, além dos relacionamentos com suas respectivas cardinalidades:

- Especialização de Usuários e Permissões:
 - Usuario: Entidade principal que possui a chave primária `id_usuario`. Mapeia atributos essenciais como nome, email, senha, telefone, receber_aviso, além de flags de controle lógico (excluido, data_exclusao, admin).
 - Herança/Especialização Total (t): A entidade Usuario se especializa em Admin (administrador da plataforma) e Comum (cliente/doador).
- Comunicação e Informativos:
 - Comunicado: Entidade identificada por `id_comunicado`. Armazena assunto, mensagem, status e data_envio. É vinculada ao usuário Admin por meio do relacionamento envia (1,1) : (0,n).
- Ciclo de Doação e Produtos:
 - Doacao: Representa os registros de doações submetidos pelos usuários através do relacionamento realiza (1,1) : (0,n). Contém os atributos `id_doacao`, nome, descricao, caminho_img, localizacao, status e data_doacao. Vincula-se a uma Categoria pelo relacionamento contém (0,n) : (1,1).
 - Produto: Entidade que armazena os itens disponíveis na loja (`id_produto`, nome, descricao, valor, caminho_img, status, excluido, data_exclusao). Conecta-se à Categoria via relacionamento contém (0,n) : (1,1) e origina-se de uma doação por meio do relacionamento torna-se (0,1) : (0,1).
- Processamento de Reservas e Vendas:
 - Compra: Entidade identificada por `id_compra` com os atributos `data_compra`, status e o atributo derivado `valor_total` (indicado em tracejado). É realizada por um Usuario pelo relacionamento realiza (1,1) : (0,n).
 - Relacionamento contém (Compra - Produto): Relacionamento N:M que conecta os produtos a uma compra com cardinalidade (0,1) : (1,n), registrando o status da transação por item.
- Entidades de Gestão de Conteúdo (Mídias Independente):
 - Banner: Identificado por `id_banner`, guarda a imagem e a ordem de exibição.
 - Instagram_Destaques: Identificado por `id_destaque`, armazena o link_post.
 - Galeria: Identificado por `id_galeria`, armazena a imagem (`caminho_img`) e o titulo_evento.

3. TECNOLOGIAS UTILIZADAS

A seleção de tecnologias para o desenvolvimento do sistema Brecharme priorizou a flexibilidade, a segurança e a rapidez no processamento de dados. A infraestrutura divide-se nas seguintes categorias:

3.1 Padrão de Arquitetura

- MVC (Model-View-Controller): A arquitetura do sistema segue o padrão MVC provido nativamente pelo framework Laravel. A camada Model é responsável pela regra de negócio e abstração do banco de dados (via ORM Eloquent); a View cuida da renderização das telas e componentes visuais; e a Controller faz a intermediação da lógica de navegação e requisições do utilizador.
- Arquitetura Baseada em Serviços e APIs (RESTful): O sistema utiliza comunicação assíncrona baseada em requisições HTTP/REST para consumo e envio de dados a APIs de terceiros (autenticação social, serviço de e-mail e mensagens de WhatsApp).

3.2 Linguagens e Tecnologias Front-End

- HTML5: Linguagem de marcação utilizada para a estruturação semântica de todas as interfaces (views) do sistema.
- CSS3 (Vanilla CSS): Recursos de estilização nativos com arquitetura modular (@import) e variáveis globais (:root), utilizados na construção de interfaces responsivas, gestão de temas, tipografia e componentes.
- JavaScript (ES6+): Linguagem client-side utilizada para manipulação do DOM, validação e aplicação de máscaras dinâmicas em formulários, controle de componentes interativos (carrosséis, menus e modais) e processamento de dados dinâmicos via data-attributes no navegador.

3.3 Linguagens, Frameworks e Infraestrutura Back-End

- PHP 8.3 ou superior: Linguagem de programação principal executada no servidor.
- Laravel Framework: Framework web em PHP responsável pela gestão de rotas, middleware de segurança, autenticação, controle de sessões e ORM Eloquent para acesso ao banco de dados.
- Node.js e npm: ambiente de execução e gerenciador de pacotes utilizados pela API de envio de mensagens de WhatsApp.
- Composer: Gestor de dependências do PHP responsável pela instalação e manutenção dos pacotes do servidor.

3.4 Sistema de Banco de Dados (SGBD)



- PostgreSQL: Sistema Gerenciador de Banco de Dados Relacional (SGBD) utilizado para o armazenamento persistente e seguro de todas as entidades da aplicação.
- PgAdmin 4: Interface gráfica para administração, criação de scripts SQL e monitorização do banco de dados PostgreSQL.

3.5 Integrações e APIs Externas

- Laravel Socialite: Pacote para implementação de autenticação OAuth2 e login social através de contas do Google.
- API de Mensagens (WhatsApp): Serviço customizado via HTTP/REST para disparo automático de notificações de status e confirmações.
- Serviço SMTP (Gmail Mailer): Protocolo para envio de emails transacionais (redefinição de senha, avisos de reservas e doações).

3.6 Ambientes e Ferramentas de Implementação e Controle

- Git e GitHub: Controle de versões e repositório centralizado do código-fonte.
- PHP Built-in Server: Servidor embutido executado via CLI (php artisan serve) para testes locais.
- FileZilla: Cliente FTP/SFTP utilizado para a transferência de ficheiros e pacotes compilados na infraestrutura de hospedagem.

4. SETUP DO SISTEMA EM PRODUÇÃO

Esta seção descreve, passo a passo, como publicar o sistema Brecharme no servidor da escola, acessível em <http://eq.projetoscti.com.br/26-brecharme>. O processo é dividido em quatro partes: cópia dos arquivos, configuração do Laravel, montagem do banco de dados PostgreSQL e publicação da API de WhatsApp.

Os comandos devem ser executados no terminal do servidor (SSH), dentro da pasta `/var/www/equipes/26-brecharme`.

4.1 Pré-Requisitos

Antes de iniciar, confirme que o servidor possui os componentes abaixo.

- PHP: Versão 8.3 ou superior, com a extensão `pdo_pgsql`
 - Para testar: rode no terminal `php -v` e `php -m | grep -i pgsql`

```
ra2457018@sftp:/var/www/equipes/26-brecharme$ php -v
PHP 8.4.23 (cli) (built: Jul 2 2026 18:43:39) (NTS)
Copyright (c) The PHP Group
Built by Ubuntu
Zend Engine v4.4.23, Copyright (c) Zend Technologies
with Zend OPcache v8.4.23, Copyright (c), by Zend Technologies
```



```
ra2457018@sftp:/var/www/equipes/26-brecharme$ php -m | grep -i pgsql
```

```
pdo_pgsql
pgsql
```

Imagem: Terminal

- Composer: Versão 2
 - Para testar: `rode composer -V`

```
ra2457018@sftp:/var/www/equipes/26-brecharme$ composer -V
```

```
Composer 2.2.6 2022-02-04 17:00:38
```

Imagem: Terminal

- [Node.js](#) e npm: Versão LTS (usados apenas pela API de WhatsApp)
 - Para testar: `rode node -v` e `npm -v`

```
ra2457018@sftp:/var/www/equipes/26-brecharme$ node -v
```

```
v20.20.2
```

```
ra2457018@sftp:/var/www/equipes/26-brecharme$ npm -v
```

```
10.8.2
```

Imagem: Terminal

Apache: Módulo `mod_rewrite` habilitado (Configurado pela infraestrutura da escola)

- PostgreSQL: Banco 26-brecharme já criado (Para conferir, utilizar as credenciais Host: `projetoscti.com.br`, Port: 54432, Database/Name: 26-brecharme, User/Role: 26-brecharme e Password: (senha do banco) acessando o pgAdmin 4 em Services -> Register -> Server...)

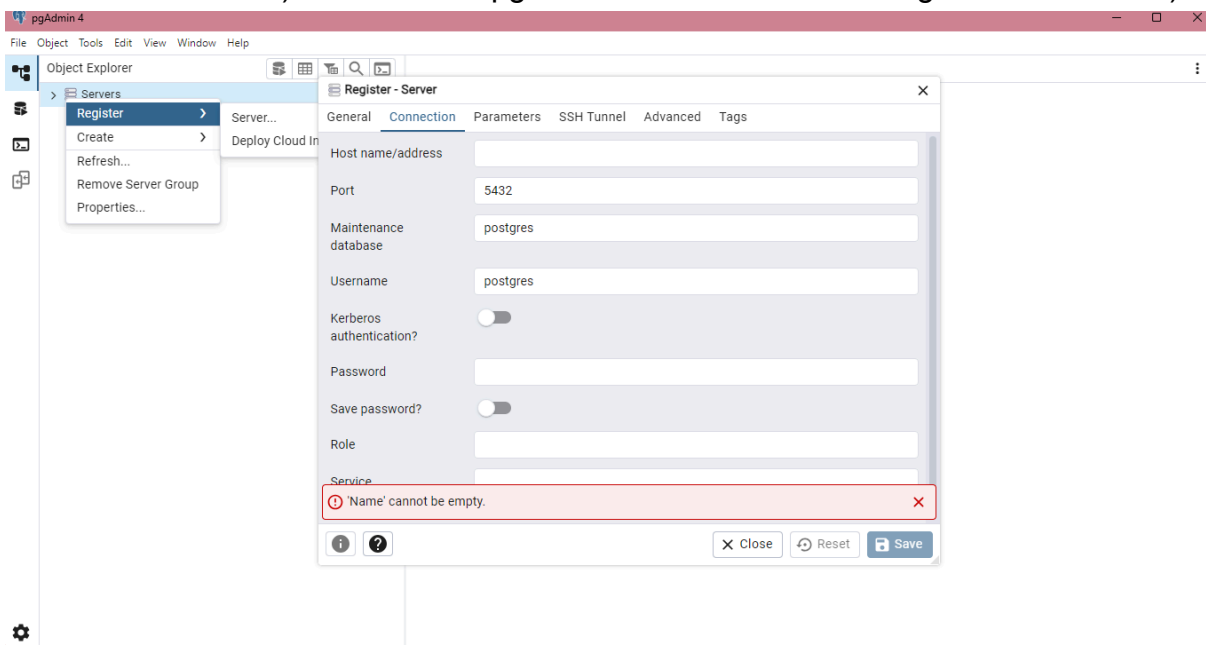


Imagem: Caminho para conectar banco

Será necessário também: o usuário e a chave SSH, a senha do banco de dados, uma senha de app do Gmail, as credenciais do Google Cloud (Client ID e Client Secret) e acesso ao celular do número de WhatsApp que enviará as mensagens, pois será necessário ler um QR Code.

4.2 Acesso ao Servidor

O acesso é feito por SFTP (para enviar arquivos) e por SSH (para executar comandos), usando a chave recebida por e-mail.

- No FileZilla, abra Arquivo > Gerenciador de Sites e clique em Novo Site.
- Em Protocolo, escolha SFTP – SSH File Transfer Protocol.
- Preencha Host com `projetoscti.com.br`, Porta com `40266` e Tipo de Logon com Arquivo de chave.
- Em Usuário, informe o seu usuário e, em Arquivo de chave, selecione a chave SSH recebida.
- Clique em Conectar.

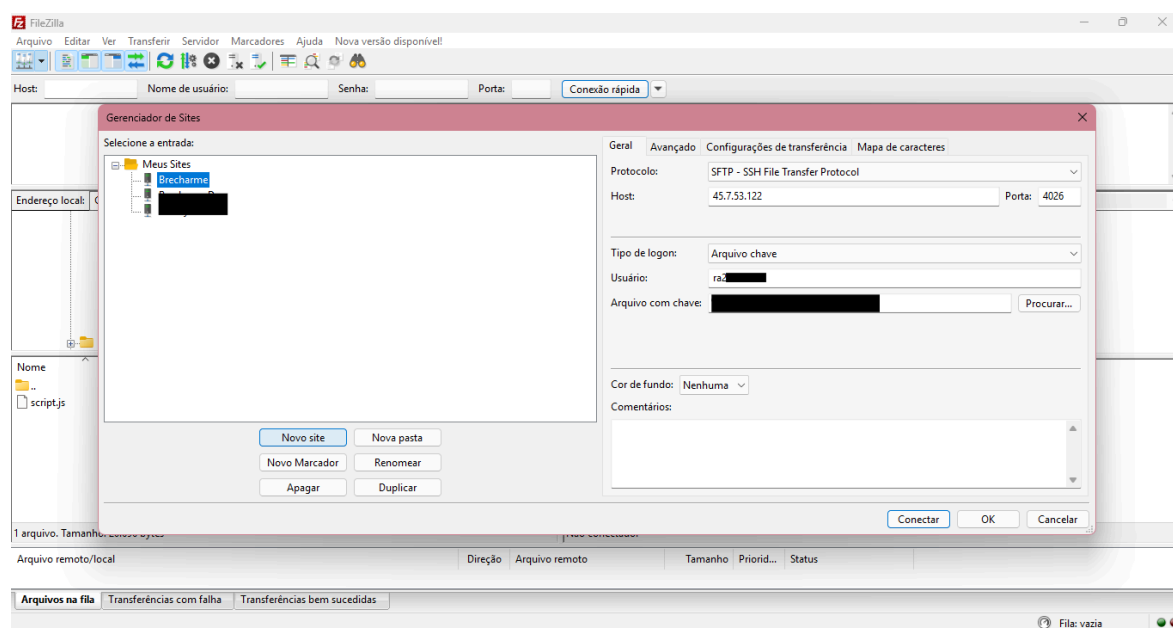


Imagem: Conexão SFTP configurada no FileZilla

- Para abrir o terminal, execute no seu computador:
`ssh -i caminho/da/chave -p 40266 SEU\_USUARIO@projetoscti.com.br`

```
PS C:\Users\nicol> ssh -i "C:\..." "ra2457018@projetoscti.com.br -p 4026"
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-190-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of qui 01 out 2026 17:49:19 -03

System load: 0.36          Processes: 249
Usage of /: 16.9% of 489.61GB Users logged in: 2
Memory usage: 64%          IPv4 address for ens18: 192.168.0.2
Swap usage: 38%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop

Manutenção de Segurança Expandida para Applications não está ativa.

46 as atualizações podem ser aplicadas imediatamente.
2 dessas atualizações são atualizações de segurança padrão.
Para ver as actualizações adicionais corre o comando: apt list --upgradable

24 atualizações de segurança adicionais podem ser aplicadas com ESM Apps.
Saiba mais sobre como ativar o serviço ESM Apps at https://ubuntu.com/esm

New release '24.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Thu Oct 1 17:09:18 2026 from 191.59.119.135
ra2457018@sftp:~$
```

Imagem: Terminal SSH conectado

4.3 Instalação das dependências PHP

Acesse a pasta do projeto e instale as dependências:

- `cd /var/www/equipes/26-brecharme`
`composer install --no-dev --optimize-autoloader`

A opção `--no-dev` ignora os pacotes usados apenas no desenvolvimento (como testes automatizados), deixando a instalação mais leve.

```
ra2457018@sftp:~$ cd /var/www/equipes/26-brecharme
ra2457018@sftp:/var/www/equipes/26-brecharme$ composer install --no-dev --optimize-autoloader
```

Imagem: Instalação das dependências com o Composer

4.4 Configurações do .env

O arquivo `.env` guarda as configurações que mudam de um ambiente para outro (banco de dados, e-mail, endereços e chaves de acesso). Para editá-lo via terminal, digite `nano .env` na raiz do projeto.

Caso o `.env` não esteja na pasta, já que ele não é clonado junto com outros arquivos do git, para criá-lo deve rodar os comandos:

- `cp .env.example .env`
- `php artisan key:generate`

As variáveis devem estar de acordo com a tabela a seguir

Variável	Valor em produção	Observação
APP_NAME	Brecharme	Nome da aplicação
APP_ENV	production	Indica ambiente de produção
APP_DEBUG	false	Ocultar detalhes de erros do público
APP_URL	http://eq.projetoscti.com.br/26-brecharme	Endereço público do sistema
DB_CONNECTION	pgsql	Tipo de banco (PostgreSQL)
DB_HOST	projetoscti.com.br	Servidor do banco
DB_PORT	54432	Porta do banco
DB_DATABASE	26-brecharme	Nome do banco
DB_USERNAME	26-brecharme	Usuário do banco
DB_PASSWORD	(senha fornecida pela escola)	Não registrar em documentos
SESSION_DRIVER, CACHE_STORE	database	Sessões e cache gravados no banco
QUEUE_CONNECTION	sync	Mensagens de WhatsApp são enviadas na hora, sem processador de fila separado
MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_ENCRYPTION	smtp, smtp.gmail.com, 587, tls	Envio de e-mails pelo Gmail
MAIL_USERNAME, MAIL_FROM_ADDRESS	brecharme.eq4@gmail.com	Conta remetente
MAIL_PASSWORD	(senha de app do Gmail)	Gerada na conta Google; não é a senha de login
WHATSAPP_API_URL	http://127.0.0.1:56828/send-message	API de WhatsApp (mesmo servidor)
WHATSAPP_NUMERO	55 + DDD + número oficial	Usado nos links de contato dos e-mails
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	(credenciais do Google Cloud)	Login com Google

Variável	Valor em produção	Observação
GOOGLE_REDIRECT_URL	http://eq.projetoscti.com.br/26-brecharme/login/google/callback	Deve ser idêntica à URI cadastrada no Google Cloud Console

4.5 Cópia dos arquivos

No servidor, os arquivos ficam no diretório `/var/www/equipes/26-brecharme`. Para ter acesso aos arquivos, basta seguir os seguintes passos:

- Rodar no terminal:
 - `git clone https://github.com/nicolecolasso/INI3A-EQ4.git /var/www/equipes/26-brecharme`
 - `cd /var/www/equipes/26-brecharme`

```
ra2457018@ftp:/var/www/equipes/26-brecharme$ ls
app      composer.json  database      phpunit.xml  resources    tests         whatsapp-api
artisan  composer.lock  index_antigo.php public        routes       vendor
bootstrap config          package.json  README.md   storage      vite.config.js
```

Imagem: Pasta com todos os arquivos e pastas

4.6 Prefixo das rotas

No servidor, o sistema é acessado dentro do caminho `/26-brecharme`. Por isso, o arquivo `routes/web.php` deve ter o grupo de prefixo ativo. Para conferência do caminho que o sistema irá seguir, abra o arquivo:

- `nano routes/web.php`
Confira se a escrita está assim:
- ```
Route::group(['prefix' => '/26-brecharme'], function () {
 // ... todas as rotas do sistema ...
});
```



```
GNU nano 6.2 routes/web.php
<?php
use Illuminate\Support\Facades\Route;

Route::group(['prefix' => '/26-brecharme'], function () {

 Route::group(['prefix' => 'admin', 'middleware' => ['auth', 'checkAdmin']], function () {
 Route::get('gerenciar', [
 'as' => 'admin.gerenciar',
 'uses' => 'App\Http\Controllers\Admin\AdminController@gerenciar'
]);

 // Banners - Painel Administrativo

 Route::get('/banners', [
 'as' => 'admin.banners',
 'uses' => 'App\Http\Controllers\Admin\BannerController@index'
]);

 Route::post('/banners/update/{ordem}', [
 'as' => 'admin.banners.update',
 'uses' => 'App\Http\Controllers\Admin\BannerController@update'
]);

 // Galeria - Painel Administrativo
 Route::get('/galeria', [
```

Imagem: Arquivo web.php configurado corretamente

#### 4.7 Criação das tabelas no Banco de Dados

O banco PostgreSQL 26-brecharme utilizado no projeto já foi criado pela escola. Restam a criação das tabelas (migrations) e o cadastro dos dados iniciais (seeders).

- Teste a conexão com o banco:
  - php artisan migrate:status
- Crie as tabelas:
  - php artisan migrate --force
- Cadastre os dados iniciais:
  - php artisan db:seed --force

Atenção: execute o db:seed apenas uma vez. Repetir o comando gera erro, pois o e-mail do administrador já estará cadastrado.

Ao final, o banco contém as seguintes tabelas:

- Usuários e acesso: users, password\_reset\_tokens e sessions.
- Doações e produtos: categorias, doacao e produto.
- Reservas: compra e produto\_reserva.
- Comunicação e conteúdo: comunicado, banners, instagram\_destaque e galerias.
- Apoio do Laravel: tabelas de cache e de fila de tarefas.

O seeder cadastra ainda: o usuário administrador (admin@brecharme.com), as categorias Roupas Masculinas, Roupas Femininas, Calçados, Acessórios e Bolsas, e os três banners padrão do carrossel.

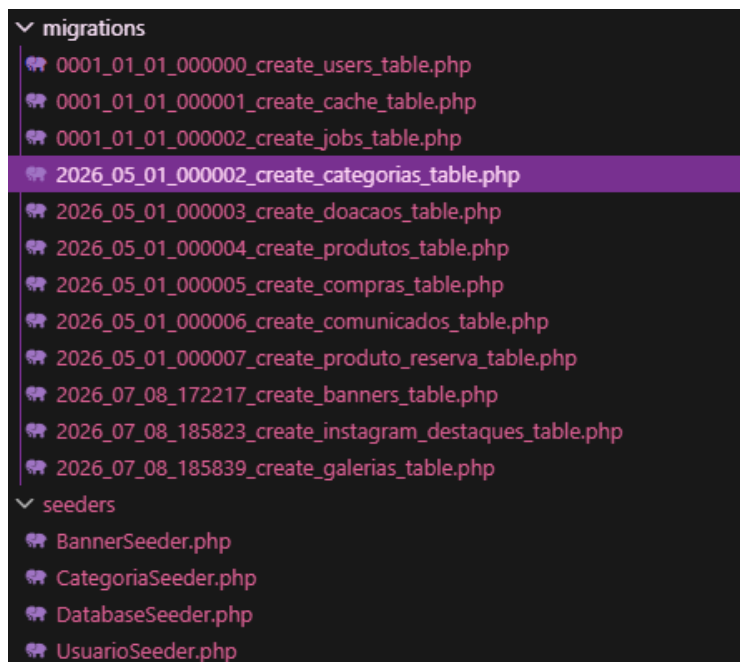


Imagem: Tabelas criadas após a migration e seeders

#### 4.8 Permissões de escrita

O sistema grava arquivos em algumas pastas: logs e arquivos temporários do Laravel (storage e bootstrap/cache) e as imagens enviadas pelos usuários e administradores, como produtos, doações, banners e galeria (public/img). O processo do servidor web precisa de permissão de escrita nelas.

No servidor atual, o usuário do servidor web (www-data) pertence ao grupo 26-brecharme, e as pastas pertencem a esse grupo com escrita liberada. Por isso, normalmente nenhum ajuste é necessário.

- Confirme os grupos do usuário do servidor web: rode o comando `id www-data`
- Verifique se o grupo 26-brecharme aparece na lista de grupos.
- Confirme as permissões das pastas: rode o comando `ls -ld storage bootstrap/cache public/img public/img/produtos`
- Verifique se cada pasta aparece com o grupo 26-brecharme e a permissão `drwxrwsr-x`, ou seja, com escrita liberada para o grupo.

Em um servidor novo, o grupo e as permissões precisam ser criados. Execute os comandos abaixo, trocando `www-data` pelo usuário do servidor web, caso seja outro:

- `cd /var/www/equipes/26-brecharme`
- `sudo mkdir -p public/img/produtos`
- `sudo chown -R www-data:www-data storage bootstrap/cache public/img`
- `sudo chmod -R 775 storage bootstrap/cache public/img`



Execute esta etapa depois de todos os comandos php artisan, pois arquivos criados por eles (como o log) pertencem ao usuário que os executou.

```
ra2457018@sftp:/var/www/equipes/26-brecharme$ id www-data
uid=33(www-data) gid=33(www-data) groups=33(www-data),1235(eq1-3a-26),1236(eq2-3a-26),1237(eq3-3a-26),1238(eq4-3a-26),1239(eq5-3a-26),1421(26-marcaai),1422(26-meuconfeito),1423(26-presco),1424(26-brecharme),1425(26-scribo),1426(26-agora),1427(26-engage),1428(26-jobee),1429(26-ecobot),1430(26-saviore),1431(26-fiscaliza),1432(26-orizon),1433(26-cloudschool),1434(loja1b),1435(loja2b),1436(loja3b),1437(loja4b),1438(loja5b),1439(loja6b),1440(loja7b),1441(loja1a),1442(loja2a),1443(loja3a),1444(loja4a),1445(loja5a),1446(loja6a),1465(loja1n),1466(loja2n),1467(loja3n),1468(loja4n)
ra2457018@sftp:/var/www/equipes/26-brecharme$ ls -ld storage bootstrap/cache public/img public/img/produtos
drwxrwsr-x 2 root 26-brecharme 4096 set 3 16:11 bootstrap/cache
drwxrwsr-x 6 root 26-brecharme 4096 ago 8 16:11 public/img
drwxrwsr-x 2 root 26-brecharme 4096 set 18 17:11 public/img/produtos
drwxrwsr-x 5 root 26-brecharme 4096 ago 8 16:11 storage
```

Imagem: Saída dos comandos id www-data e ls -ld storage bootstrap/cache public/img public/img/produtos

## 4.9 Configurações do Apache

O Apache deve servir a pasta do sistema de forma que o endereço <http://eq.projetoscti.com.br/26-brecharme> aponte para `/var/www/equipes/26-brecharme`. Para isso, são necessários:

- O módulo `mod_rewrite` habilitado (`sudo a2enmod rewrite`).
- A opção `AllowOverride All` para a pasta, para que os arquivos `.htaccess` sejam lidos.

O arquivo `.htaccess`, localizado na raiz do projeto, direciona todas as requisições para a pasta `public`, que é a única exposta ao navegador:

```
<IfModule mod_rewrite.c>
 RewriteEngine On
 RewriteCond %{REQUEST_URI} !^/public/
 RewriteRule ^(.*)$ public/$1 [L]
</IfModule>
```

Após qualquer alteração na configuração do Apache, teste e recarregue o serviço:

- `sudo apache2ctl configtest`
- `sudo systemctl reload apache2`

Acesse <http://eq.projetoscti.com.br/26-brecharme> e confirme que a página inicial é exibida.



```
GNU nano 6.2 .htaccess
<IfModule mod_rewrite.c>
 RewriteEngine On
 RewriteCond %{REQUEST_URI} !^/public/
 RewriteRule ^(.*)$ public/$1 [L]
</IfModule>

[Read 5 lines (Converted from DOS format)]
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-6 Copy
```

Imagem: Arquivo .htaccess

#### 4.10 Publicação da API de Whatsapp

O envio de mensagens de WhatsApp é feito por um serviço separado, desenvolvido em Node.js (Express e whatsapp-web.js). Ele escuta na porta 56828 e recebe do Laravel, pela rota POST /send-message, o telefone e o texto a serem enviados. O endereço do serviço é informado ao sistema pela variável `WHATSAPP_API_URL`.

- Acesse a pasta do serviço e instale as dependências:
  - `cd /var/www/equipes/26-brecharme/whatsapp-api`
  - `npm install`
- Inicie o serviço manualmente, para conectar o WhatsApp:
  - `node index.js`
- Um QR Code será exibido no terminal. No celular do número oficial do brechó, abra o WhatsApp, acesse Aparelhos conectados > Conectar um aparelho e leia o código.
- Aguarde a mensagem Cliente do WhatsApp conectado e pronto para enviar mensagens! e interrompa o serviço com `Ctrl+C`.
- Inicie o serviço de forma permanente com o PM2:
  - `pm2 start index.js --name whatsapp-api`
  - `pm2 save`
  - `pm2 startup`
- O comando `pm2 startup` exibe uma linha de comando começando com `sudo`. Copie e execute essa linha, para que o serviço volte a funcionar quando o servidor for reiniciado.
- Confirme que o serviço está ativo:
  - `pm2 list`



A sessão do WhatsApp fica salva na pasta `.wwebjs_auth`, portanto o QR Code só é pedido na primeira vez. Essa pasta não deve ser apagada. Ao restaurar um backup, copie-a de volta para evitar uma nova leitura do QR Code. Mantenha apenas uma instância do serviço ativa, usando somente a pasta oficial indicada acima.

Segurança: a API não possui autenticação e deve ser acessada apenas pelo Laravel, no mesmo servidor. Mantenha a porta 56828 bloqueada no firewall para acessos externos.

Como a variável `QUEUE_CONNECTION` está configurada como `sync`, o Laravel envia cada mensagem no mesmo momento em que o administrador do sistema envia o comunicado, sem precisar de outro processo além do `whatsapp-api`.

## 5. SETUP DE DESENVOLVIMENTO

Esta seção orienta um novo desenvolvedor a configurar o ambiente local para executar, alterar e testar o sistema Brecharme no próprio computador. Ao final, o sistema estará acessível em `http://localhost:8000` e a API de WhatsApp estará conectada a um número de teste.

O processo tem seis partes: instalação das ferramentas, obtenção do código, criação do banco de dados, configuração do arquivo `.env`, execução da API de WhatsApp e execução do sistema.

### 5.1 Pré-requisitos

As ferramentas a seguir devem estar instaladas no computador:

- Git: Qualquer versão recente (Para testar: `git --version`)

```
C:\Users\nicol>git --version
git version 2.48.1.windows.1
```

Imagem: Versões das ferramentas instaladas no computador

- PHP: Versão 8.3 ou superior, com a extensão `pdo_pgsql` habilitada (Para testar: `php -v` e `php -m | findstr /I pgsql`)

```
C:\Users\nicol>php -v
PHP 8.4.4 (cli) (built: Feb 11 2025 16:24:56) (NTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Zend Engine v4.4.4, Copyright (c) Zend Technologies

C:\Users\nicol>php -m | findstr /I pgsql
pdo_pgsql
pgsql
```

Imagem: Versões das ferramentas instaladas no computador

- Composer: versão 2 (Para testar `composer -V`)



```
C:\Users\nicol>composer -V
Composer version 2.8.5 2025-01-21 15:23:40
PHP version 8.4.4 (C:\Program Files\PHP\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
```

Imagem: Versões das ferramentas instaladas no computador

- PostgreSQL e pgAdmin 4: Versão recente. O sistema só funciona com PostgreSQL (Para testar psql --version )

```
C:\Program Files\PostgreSQL\17\bin>psql --version
psql (PostgreSQL) 17.4
```

Imagem: Versões das ferramentas instaladas no computador

- Node.js e npm: Versão LTS (usados apenas pela API de WhatsApp) (Para testar node -v e npm -v )

```
C:\>node -v
v22.20.0

C:\> npm -v
10.9.3
```

Imagem: Versões das ferramentas instaladas no computador

- VS Code (sugestão)

Saiba também: o endereço do repositório no GitHub, acesso ao celular de um número de WhatsApp para testes, a senha de app do Gmail usada pelo sistema (solicitar à equipe) e as credenciais do login com Google (Client ID e Client Secret).

## 5.2 Obtenção do código

Escolha uma pasta de trabalho e clone o repositório:

- git clone <https://github.com/nicolecolasso/INI3A-EQ4.git> brecharme
- cd brecharme

Abra a pasta no editor de código e confirme que existem as pastas como app, database, public, routes e whatsapp-api.

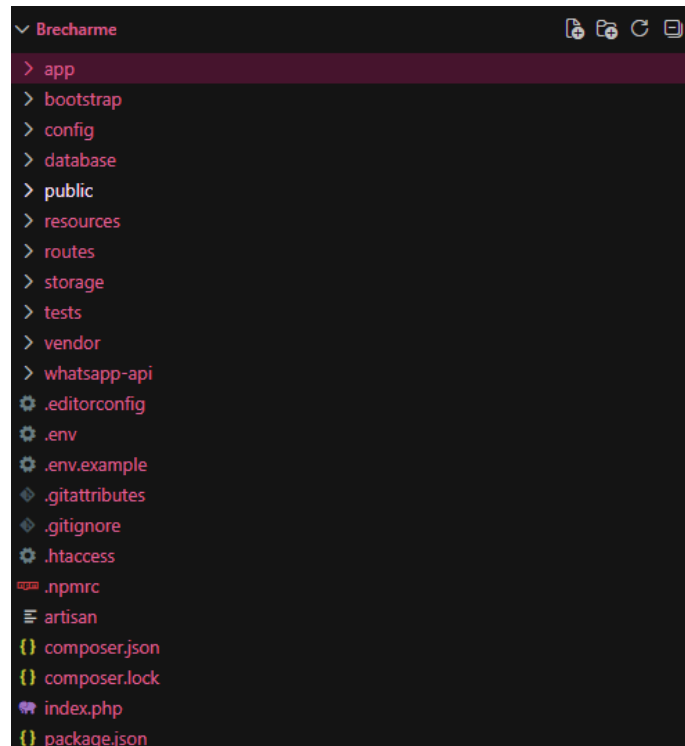


Imagem: Estrutura de pastas

### 5.3 Instalação das dependências PHP

Na pasta do projeto, rode o comando para instalar: `composer install`

```
C:\INI3A-EQ4\src\Brecharme>composer install|
```

Imagem: Terminal

### 5.4 Criação do banco de dados local

O banco de dados local deve se chamar brecharme. Ele pode ser criado pelo pgAdmin 4.

- Conecte-se ao servidor PostgreSQL local, com o usuário postgres e a senha definida na instalação.
- Clique com o botão direito em Databases > Create > Database.
- Informe o nome brecharme e clique em Save.

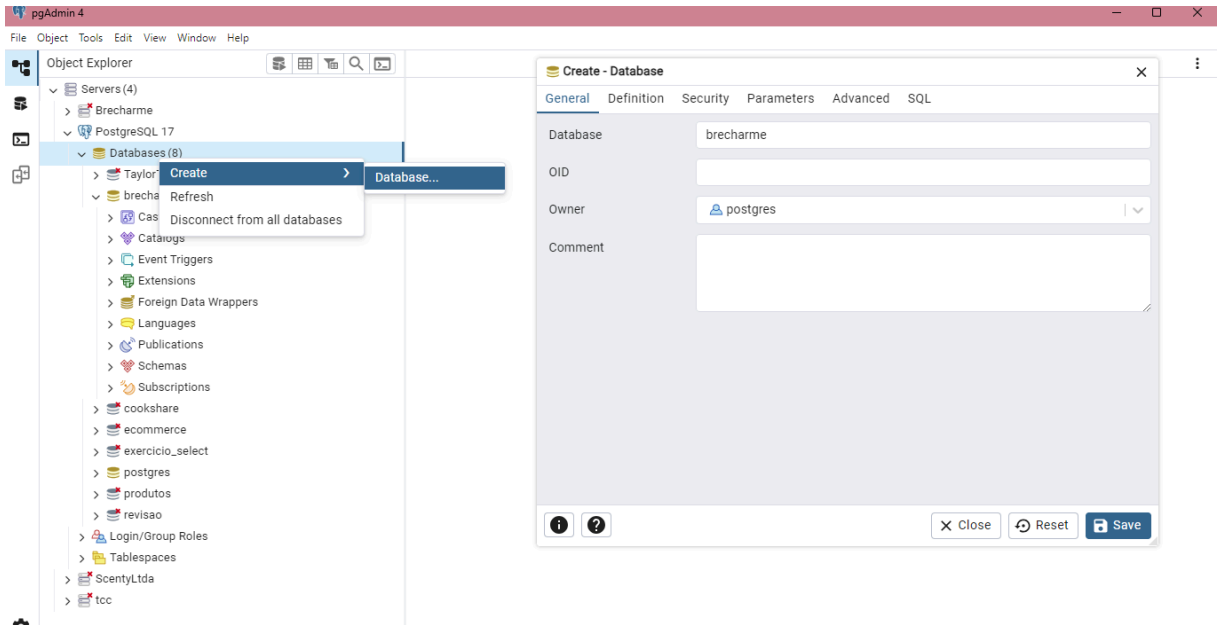


Imagem: Banco de dados brecharme criado no PostgreSQL local

## 5.5 Configurações do arquivo .env

Crie o arquivo a partir do modelo e gere a chave da aplicação, já que ele não vem com o git:

- `cp .env.example .env`
- `php artisan key:generate`

Abra o arquivo .env no editor e preencha as variáveis conforme a tabela abaixo.

Limpe o cache de configuração:

- `php artisan config:clear`

| Variável      | Valor em desenvolvimento    | Observação                                      |
|---------------|-----------------------------|-------------------------------------------------|
| APP_ENV       | local                       | Ambiente de desenvolvimento                     |
| APP_DEBUG     | true                        | Exibe detalhes dos erros, útil para desenvolver |
| APP_URL       | http://localhost:8000       | Endereço do servidor local                      |
| DB_CONNECTION | pgsql                       | Tipo de banco (PostgreSQL)                      |
| DB_HOST       | 127.0.0.1                   | Banco no próprio computador                     |
| DB_PORT       | 5432                        | Porta padrão do PostgreSQL                      |
| DB_DATABASE   | brecharme                   | Banco criado na etapa 5.4                       |
| DB_USERNAME   | postgres                    | Usuário do PostgreSQL local                     |
| DB_PASSWORD   | (senha do PostgreSQL local) | Definida na instalação do PostgreSQL            |

|                                             |                                             |                                                                                      |
|---------------------------------------------|---------------------------------------------|--------------------------------------------------------------------------------------|
| SESSION_DRIVER,<br>CACHE_STORE              | database                                    | Sessões e cache gravados no banco                                                    |
| QUEUE_CONNECTION                            | sync                                        | Mensagens de WhatsApp são enviadas na hora, sem processador de fila                  |
| MAIL_MAILER                                 | smtp                                        | Use log para gravar os e-mails em storage/logs/laravel.log, sem enviá-los de verdade |
| MAIL_HOST,<br>MAIL_PORT,<br>MAIL_ENCRYPTION | smtp.gmail.com, 587, tls                    | Envio pelo Gmail (quando MAIL_MAILER=smtp)                                           |
| MAIL_USERNAME,<br>MAIL_FROM_ADDRESS         | brecharme.eq4@gmail.com                     | Conta remetente                                                                      |
| MAIL_PASSWORD                               | (senha de app do Gmail)                     | Solicitar à equipe                                                                   |
| WHATSAPP_API_URL                            | http://127.0.0.1:56828/send-message         | API de WhatsApp local (etapa 5.7)                                                    |
| WHATSAPP_NUMERO                             | 55 + DDD + número de teste                  | Usado nos links de contato dos e-mails                                               |
| GOOGLE_CLIENT_ID,<br>GOOGLE_CLIENT_SECRET   | (credenciais do Google Cloud)               | Login com Google                                                                     |
| GOOGLE_REDIRECT_URL                         | http://localhost:8000/login/google/callback | Deve ser idêntica à URI cadastrada no Google Cloud Console                           |

No ambiente local, as duas linhas do prefixo /26-brecharme no arquivo routes/web.php devem ser comentadas. Por isso, o sistema é acessado diretamente em http://localhost:8000, sem o prefixo. Não envie o arquivo .env para o Git.

## 5.6 Criação das tabelas e dados iniciais

Para criar as tabelas já definidas, rode:

- php artisan migrate

Cadastre os dados iniciais:

- php artisan db:seed

O comando db:seed cadastra o usuário administrador (admin@brecharme.com, com a senha definida no arquivo database/seeder/UsuarioSeeder.php), as categorias Roupas Masculinas, Roupas Femininas, Calçados, Acessórios e Bolsas, e os três banners padrão do carrossel.

Atenção: execute o db:seed apenas uma vez. Para apagar tudo e recomeçar do zero, use php artisan migrate:fresh --seed. Esse comando apaga todos os dados do banco e deve ser usado apenas em desenvolvimento.

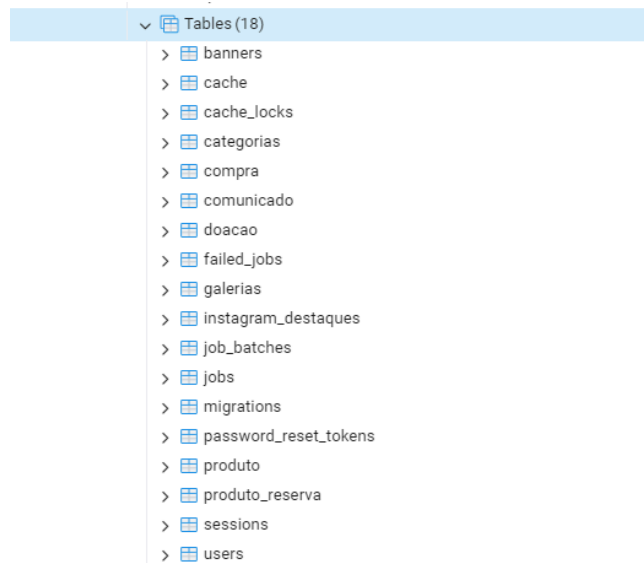


Imagem: Tabelas criadas no PostgreSQL local

## 5.7 Execução da API de Whatsapp

A API de WhatsApp é um serviço Node.js separado, localizado na pasta whatsapp-api. O Laravel envia a ela o telefone e a mensagem pela rota POST /send-message, na porta 56828.

Em um novo terminal, acesse a pasta e instale as dependências:

- `cd whatsapp-api`
- `npm install`

Inicie o serviço:

- `node index.js`

Um QR Code será exibido no terminal. No celular do número de teste, abra o WhatsApp, acesse Aparelhos conectados > Conectar um aparelho e leia o código.

Aguarde a mensagem “Cliente do WhatsApp” conectado e “Pronto para enviar mensagens!”

A instalação das dependências baixa também um navegador (Chromium), usado pela biblioteca para operar o WhatsApp Web, e por isso exige conexão com a internet. A sessão fica salva na pasta `.wwebjs_auth`, portanto o QR Code só é pedido na primeira vez. Essa pasta contém a sessão do WhatsApp e não deve ser enviada ao Git.

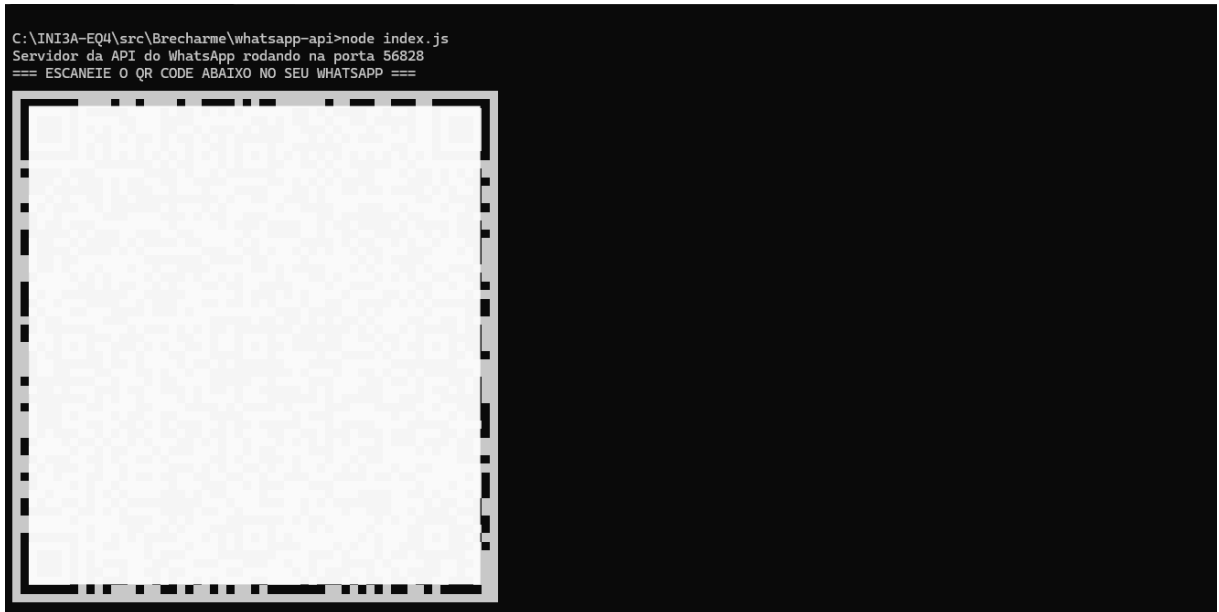


Imagem: Terminal com o QR Code

## 5.8 Execução do sistema

Com o banco criado e a API de WhatsApp ativa, inicie o servidor do Laravel em outro terminal, na pasta do projeto:

- `php artisan serve`

O sistema ficará disponível em `http://localhost:8000`. Mantenha dois terminais abertos: um com o `php artisan serve` e outro com o `node index.js` da API de WhatsApp.

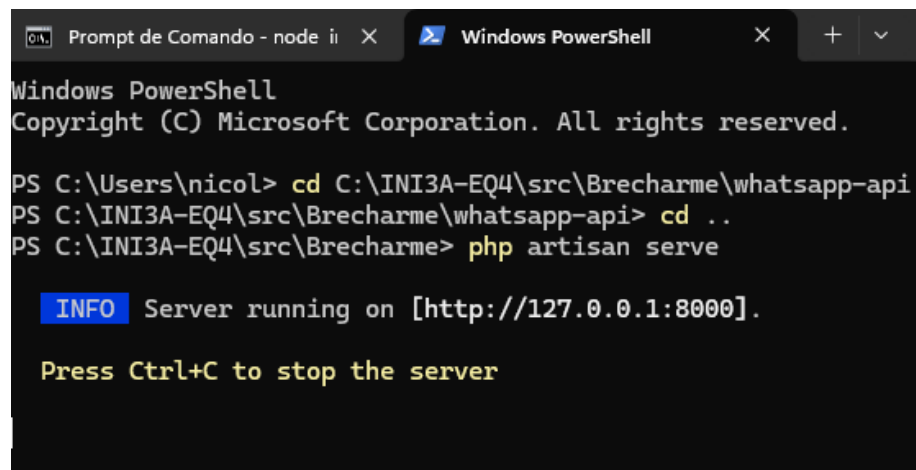


Imagem: Terminal com `php artisan serve` e `node index.js`

## 5.9 Estrutura do projeto

| Pasta ou arquivo | Conteúdo |
|------------------|----------|
|------------------|----------|

|                                        |                                                                                                                |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------|
| app/Http/Controllers                   | Controladores das telas públicas (login, vitrine, carrinho, perfil e páginas institucionais)                   |
| app/Http/Controllers/Admin             | Controladores do painel administrativo (produtos, usuários, reservas, doações, banners, galeria e comunicados) |
| app/Models                             | Modelos que representam as tabelas do banco, como Produto, Compra, Categoria e Banner                          |
| app/Jobs                               | Tarefas do sistema, como EnviarWhatsAppJob, que envia mensagens pela API de WhatsApp                           |
| routes/web.php                         | Todas as rotas do sistema                                                                                      |
| resources/views                        | Telas do sistema (templates Blade)                                                                             |
| database/migrations e database/seeders | Criação das tabelas e dados iniciais                                                                           |
| public                                 | Arquivos estáticos: css, js, fonts e img (inclui as imagens enviadas, como img/produtos)                       |
| whatsapp-api                           | Serviço Node.js de envio de mensagens de WhatsApp                                                              |
| .env                                   | Configurações do ambiente. Não é enviado ao Git.                                                               |